

Ayudantía 1

Repaso de Python

Ayudante: Enzo Vásquez C.

Profesor: Felipe Gómez



Universidad de O'Higgins
Instituto de Ciencias de la Ingeniería

4 de septiembre de 2026

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores
 - Try-Except y Raise

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores
 - Try-Except y Raise
- 8 Ejercicio
 - Gestor de inventario

Código 1: Asignación múltiple e intercambio

```
1 a, b, c = 1, 2, 3
2 x, y = y, x
```

Tipos de Datos

- `int`: enteros; `float`: reales.
- `bool`: `True` o `False`.
- `None`: ausencia de valor.
- `str`: cadenas de caracteres.

Operadores aritméticos, lógicos y más

- Aritméticos: `+` `-` `*` `/` `//` `%` `**`.
- Comparación: `==` `!=` `<` `<=` `>` `>=`.
- Lógicos: `not`, `and`, `or`.
- Pertenencia: `in`, `not in`
- Identidad: `is`, `is not`.

Precedencia

El orden de operaciones sigue a: paréntesis, potencia, multiplicación, suma, comparación y lógica.

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 **Strings**
 - **Operaciones con texto**
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores
 - Try-Except y Raise
- 8 Ejercicio
 - Gestor de inventario

- Un string es una secuencia inmutable de caracteres.
- Los índices comienzan en 0; los negativos cuentan desde el final (esto también aplica para colecciones).
- El slicing se usa como `string[inicio:fin:paso]`. Se puede solo usar `string[inicio:fin]` (por defecto paso 1).

1

¹ Existe muchos métodos para los strings, ver en: <https://docs.python.org/3/library/stdtypes.html>

Código 2: Slicing y métodos de strings

```
1 texto = "Programacion"
2 print(texto[0:4], texto[:4], texto[4:])
3 print(texto[:2], texto[::-1])
4
5 limpio = texto.strip().lower()
6 partes = "Ana,22,Rancagua".split(",")
7 frase = " ".join(["Python", "es", "legible"])
8
```

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 **Estructuras de datos**
 - **Comparación de colecciones**
 - **Sets**
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores
 - Try-Except y Raise
- 8 Ejercicio
 - Gestor de inventario

Elegir una colección

- **Lista:** ordenada, mutable, permite duplicados y acceso por índice.
- **Tupla:** ordenada, inmutable, permite duplicados y acceso por índice.
- **Diccionario:** mutable, con claves únicas y acceso por clave.
- **Set:** mutable, sin duplicados y útil para pertenencia.

Listas anidadas y comprehensions

Código 3: Matriz y filtros

```
1 matriz = [[1, 2, 3],  
2           [4, 5, 6],  
3           [7, 8, 9]]  
4 print(matriz[1][2])  
5  
6 cuadrados = [x**2 for x in range(10)]  
7 pares = [x for x in range(20) if x % 2 == 0]
```

Código 4: Unicidad y operaciones de conjuntos

```
1 a = {1, 2, 3, 3, 3}
2 vacio = set()
3
4 valores = {1, 2, 3}
5 valores.discard(2)
6 valores.remove(3)
7
8 a, b = {1, 2, 3, 4}, {3, 4, 5, 6}
9 union = a | b
10 interseccion = a & b
11 diferencia = a - b
12 simetrica = a ^ b
```

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 **Control de flujo**
 - **Condicionales**
 - **Ciclos for**
 - **Ciclos while**
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores
 - Try-Except y Raise
- 8 Ejercicio
 - Gestor de inventario

- Python ejecuta la primera condición verdadera.
- Una condición general antes que una específica puede volverla inalcanzable.

Código 5: if, elif, else y expresión condicional

```
1 nota = 6.8
2 if nota >= 6.0:
3     print("Excelente")
4 elif nota >= 4.0:
5     print("Aprobado")
6 else:
7     print("Reprobado")
8
9 estado = "adulto" if edad >= 18 else "menor"
```


- Use `for` para recorrer un iterable o un rango conocido.
- Preferir `for dato in datos` si no necesita el índice.
- `range` no incluye su límite final.
- `break` termina y `continue` salta una iteración.
- El `else` de un `for` se ejecuta si no ocurrió un `break`.

Código 6: enumerate y zip

```
1 for ciudad in ["Rancagua", "Santiago"]:
2     print(ciudad)
3
4 for i, ciudad in enumerate(ciudades, start=1):
5     print(i, ciudad)
6
7 for nombre, nota in zip(nombres, notas):
8     print(nombre, nota)
```

Búsqueda con for...else

Código 7: Búsqueda y break

```
1 for numero in numeros:
2     if numero == objetivo:
3         print("Encontrado")
4         break
5 else:
6     print("No encontrado")
```

- Use `while` cuando se repita una acción hasta que cambie una condición.
- La variable que controla la condición debe modificarse dentro del ciclo.
- Verifique que exista una salida para evitar un ciclo infinito.

Contador con while

Código 8: Condición y actualización

```
1 contador = 0
2 while contador < 5:
3     print(contador)
4     contador += 1
```

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores
 - Try-Except y Raise
- 8 Ejercicio
 - Gestor de inventario

Ejercicio 1

- a) Mostrar los múltiplos de un número entre 1 y 100.
- b) Calcular el factorial de un número usando un ciclo `for`.
- c) Pedir un número entero positivo mayor que 2 y mostrar si es primo.

Ejercicio 2

Crear un programa que calcule el dígito verificador de un RUT usando el algoritmo del Módulo 11:

1. Seleccionar los números del RUT sin el dígito verificador.
2. Reordenarlos de derecha a izquierda.
3. Multiplicarlos repetidamente por 2, 3, 4, 5, 6, 7.
4. Sumar los resultados y calcular el módulo 11.
5. Restar el resto a 11 para obtener el dígito.

Importante

Si el resultado es 11, el dígito es 0. Si es 10, el dígito es la letra K.

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 **Funciones**
 - **Funciones**
- 7 Manejo de errores
 - Try-Except y Raise
- 8 Ejercicio
 - Gestor de inventario

Cómo definir, parámetros y scope

- Una función nos permite reutilizar código, facilitar lectura y crear un código sostenible.
- En Python, una función por defecto devuelve `None`.
- Una variable creada dentro de una función posee un scope (alcance) local.
- Una función puede tener parámetros, parámetros por defectos y cantidad de parámetros indefinidos.

Código 9: Tipos de funciones

```
1 def suma(*args):  
2     return sum(args)  
3  
4 def area_circulo(pi=3.14, radio):  
5     return pi*radio**2  
6  
7 def main():  
8     a = suma(1,2,3,4,5,6,7,8,9,10)  
9     print(suma)  
10  
11    b = area_circulo(10)
```

Contenidos

- 1 Python esencial
 - Variables y tipos
 - Operadores aritméticos, lógicos y más
- 2 Strings
 - Operaciones con texto
- 3 Estructuras de datos
 - Comparación de colecciones
 - Sets
- 4 Control de flujo
 - Condicionales
 - Ciclos for
 - Ciclos while
- 5 Ejercicios
 - Ejercicio 1
 - Ejercicio 2
- 6 Funciones
 - Funciones
- 7 Manejo de errores**
 - Try-Except y Raise**
- 8 Ejercicio
 - Gestor de inventario

Try-Except y Raise

Código 10: Uso de try-except

```
1 a = 5; b = 0
2 try:
3     c = a/b
4 except ZeroDivisionError:
5     print("No se ha podido realizar la división")
```

Código 11: Uso de Raise

```
1 a = 5
2 b = int(input("Ingreso un número distinto de 0: "))
3
4 if b == 0:
5     raise ZeroDivisionError("Error, no se puede dividir por 0!")
6
7 c = a/b # esto ya no se ejecuta!
```

La empresa M (experta en distribución de videojuegos) desea digitalizar el inventario de una sucursal utilizando Python. El sistema debe ser modular, escalable y a prueba de errores humanos. La única instrucción de la empresa es que al menos se implemente funciones de buscar, modificar (como vender) y agregar.

Considere que los productos se guardan en un diccionario de la forma {"nombre":

↪ {"precio" : 123, "stock" : 123}}.